

CAP 931: Capstone - Build a Sales Agent Prototype Using Multi-Agent GPT Models

1. Project Title & Overview

- **Project Name:** Multi-Agent Sales Assistant Prototype ([sales-agent-prototype](#))
 - **Course/Capstone:** CAP 931 Capstone Project
 - **Author:** Franky Achu Ndalle Jr.
 - **Objective:** Build an intelligent multi-agent sales assistant prototype powered by Large Language Models and Streamlit to help enterprise sales representatives analyze target accounts, evaluate competitor overlaps, and generate executive-ready account insight one-pagers.
-

2. Technical Foundation & Environment Setup (Step 1)

To establish a robust development environment for rapid prototyping, the following technical stack and local configurations were implemented:

- **Programming Language:** Python 3.10+
- **Frameworks & Libraries:**
 1. [streamlit](#) (For front-end interactive UI development)
 2. [langchain-groq](#) & [langchain-core](#) (For orchestrating multi-agent chains and prompt templates)
 3. [python-dotenv](#) (For secure environment variable handling)
- **Environment Initialization Commands:**

PowerShell:

1. `mkdir sales-agent-prototype`
2. `cd sales-agent-prototype`
3. `pip install streamlit langchain langchain-groq python-dotenv`

- **Security Configuration:** API keys were isolated via a local `.env` file (`GROQ_API_KEY`).
 - **UI Initiation:** Tested and launched locally via `streamlit run main.py`, successfully compiling the server at <http://localhost:8502/>.
-

3. Dynamic Inputs & Data Collection Layer (Step 2)

The user interface features a reactive Streamlit sidebar that accepts configurable account metadata, allowing sales representatives to dynamically input campaign parameters:

- **Product Name:** The software/solution being pitched (e.g., Enterprise Cloud Data Platform).
 - **Value Proposition:** Core advantage statement (e.g., Reduces data warehousing costs by 40% with automated scaling).
 - **Target Company URL:** The prospective account domain (e.g., <https://example-enterprise.com>).
 - **Product Category:** Industry or technical classification (e.g., Data Warehousing).
 - **Competitor Names / URLs:** Market rivals for comparative evaluation (e.g., Snowflake, Databricks).
 - **Target Customer / Persona:** Decision-maker role (e.g., Chief Data Officer).
-

4. LLM Capabilities, Model Selection & Data Integration (Step 3)

- **Model Engine Selected:** Groq API running `llama-3.3-70b-versatile` via LangChain's `ChatGroq` wrapper.
- **Technical Justification:**
 1. **Speed:** Groq's high-performance inference engine provides near-instantaneous token generation, enabling seamless multi-agent prompt chaining without latency.
 2. **Reasoning Quality:** The 70B parameter model offers advanced contextual reasoning, structured parsing, and strategic synthesis comparable to proprietary GPT-4 architectures.
- **Data Integration & Context Refinement:**

1. The system directly integrates the user-provided input URLs and context variables (`{target_url}`, `{product_name}`, `{category}`, `{competitors}`) into the LLM system instructions and prompt templates.
 2. This ensures that the generated model outputs are precisely grounded in the target account's specific data points, guaranteeing accuracy and operational relevance during downstream agent execution.
- **Prompt Chaining Architecture (`main.py`):**
 1. Sales Intake Agent: Parses target account requests to identify core customer goals, use cases, and technical requirements.
 2. Market Research Agent: Evaluates strategic positioning and direct competitor overlaps against specified rivals using prior intake context.
 3. Strategy Synthesis Agent: Consolidates prior agent outputs into a unified executive one-pager.
-

5. System Outputs & Executive One-Pager Deliverable (Step 4 & 5)

When executed through the Streamlit interface, the application generates a structured one-pager aligned with Per Scholas evaluation requirements:

- **Company Strategy:** Summarizes organizational activities, inferred technology stacks, and market focus areas.
- **Competitor Mentions:** Evaluates direct overlaps against specified rivals (e.g., Snowflake, Databricks).
- **Leadership Information:** Identifies key executive focus areas across product development, sales, and customer success.
- **Tailored Pitch Strategy:** Delivers actionable messaging emphasizing total cost of ownership reduction and automated scaling.

Sample System Output

Account Insight Summary:

Executive Summary: Example Enterprise is seeking to pitch an Enterprise Cloud Data Platform to the Chief Data Officer at example-enterprise.com. The customer goal is to manage large amounts of data to support business decision-making, focusing on scalability, security, and efficiency. Key requirements include scalability, data governance compliance, and high-performance processing capabilities.

Competitor Analysis: Competes directly with Snowflake and Databricks in the cloud data platform market. Differentiators include simplified cost structures, automated scaling, and a 40% reduction in data warehousing expenses.

Key Leadership Focus Areas:

- **Product Development:** Expanding cloud data platform capabilities and performance.
- **Customer Success:** Realizing cost savings and operational efficiency outcomes.

Tailored Pitch Strategy: Emphasize total cost of ownership reduction, ease of use through automated scaling, and proven performance capabilities to position the platform as a superior alternative for enterprise data management.

6. Optional Enhancements

- **Improving Output Results:** Utilizing multi-agent prompt chaining ([intake_output](#) → [research_output](#) → [synthesis_output](#)) to isolate agent responsibilities, reduce hallucination, and enhance structural depth.
- **Alert System:** Designed a background cron-job/Celery architecture to scrape target RSS feeds or job boards using keywords and dispatch automated email notifications via [smtplib](#) / SendGrid.
- **Production Deployment:** Containerized via Docker and deployed to cloud environments (AWS ECS or Streamlit Cloud), utilizing Redis/Celery for asynchronous processing queues and secure environment secret managers.

- **Advanced Model Routing:** Integrating multimodal models for visual data processing, automated slide-deck generation (`python-pptx`), and Retrieval-Augmented Generation (RAG) via vector databases (ChromaDB) for proprietary document ingestion.
-

7. Documentation

A. Technical Documentation & Code Structure (`main.py`)

B. Time Management Allocation

- **Phase 1 (Environment Setup & UI Initiation - ~20%):** Configuring Python environments, installing Streamlit, and setting up secure local API keys.
- **Phase 2 (Prompt Engineering & Multi-Agent Chaining - ~50%):** Designing specialized agent roles and linking upstream variables to downstream prompt templates.
- **Phase 3 (Testing, Sandboxing & Documentation - ~30%):** Validating local execution (`streamlit run main.py`), building test chat scripts, and formatting project documentation.

C. Challenges and Solutions

- **Challenge:** Managing context hand-off between independent LLM agent prompts without losing track of specific user parameters or competitor data.
- **Solution:** Implemented structured sequential variable passing in `main.py`, where downstream chains explicitly require upstream outputs (`intake_output` and `research_output`).

D. Experiments & Outcomes

- **Experiment:** Tested a single monolithic prompt versus a multi-agent prompt-chaining architecture.
- **Outcome:** Monolithic prompts occasionally mixed market strategy with technical specs. Splitting logic into dedicated autonomous agent roles dramatically improved output clarity and professional formatting.